

Sprawozdanie z etapu 3.

Szymon Hładyszewski, Wiktor Koczkodaj, Jakub Kogut, Wojciech Typer

22 listopada 2025

W tym etapie projektu przedstawiamy nasz plan na jego implementację. Składa się ona z odpowiednio z projektu backendu oraz frontendu.

1 Backend

Nasz backend będzie odpowiedzialny za przechowywanie oraz przetwarzanie wszelakich danych jak i musi posiadać *user friendly* metodę na ich wprowadzenie. W tym celu zdecydowaliśmy się na użycie frameworku **Django** wraz z **Django REST Framework** do stworzenia API. Do przechowywania danych użyjemy bazy danych **PostgreSQL**, która jest utrzymywana przez **Django**.

1.1 Baza danych

W bazie danych przechowywane będą wszelkie dane dotyczące użytkowników jak i zdobytych przez nich nagród, czy dane do kursów oraz quizów, a także forum.

1.1.1 Tabele

W bazie danych znajdować się będą następujące tabele:

- **User:** przechowuje dane użytkowników
 - **id (PK)** - unikalny identyfikator użytkownika
 - **nickname** - nazwa użytkownika *unique*
 - **password_hash** - zahashowane hasło użytkownika
 - **avatar_id** - identyfikator avatara użytkownika (możliwe rozszerzenie bazy o tabelę Avatar)
- **Course:** zawiera kursy dostępne w serwisie
 - **id (PK)** - unikalny identyfikator kursu
 - **name** - tytuł kursu
- **Lesson:** reprezentuje lekcje, z których składają się kursy
 - **id (PK)** - unikalny identyfikator lekcji
 - **name** - tytuł lekcji
- **Slide:** reprezentuje slajdy w lekcjach
 - **id (PK)** - unikalny identyfikator slajdu
 - **lesson_id (FK)** - identyfikator lekcji, do której należy slajd
 - **text_content** - treść slajdu
 - **order** - kolejność slajdu w lekcji
 - **image_url** - (opcjonalnie) URL do obrazka na slajdzie
- **Quiz:**
 - **id (PK)** - unikalny identyfikator quizu

- name - tytuł quizu
- exp_weight - waga doświadczenia za poprawne rozwiązanie quizu
- **Question:** pytania w quizach
 - id (PK) - unikalny identyfikator pytania
 - quiz_id (FK) - identyfikator quizu, do którego należy pytanie
 - question_type - typ pytania (otwarte/zamknięte)
 - content - treść pytania
- **AnswerPattern:** wzorce odpowiedzi dla otwartych pytań (wartości numeryczne)
 - id (PK) - unikalny identyfikator wzorca odpowiedzi
 - question_id (FK) - identyfikator pytania, do którego należy wzorzec
 - pattern - wzorzec odpowiedzi (np. zakres wartości)
- **AnswerOption:** możliwe odpowiedzi dla pytań zamkniętych
 - id (PK) - unikalny identyfikator opcji odpowiedzi
 - question_id (FK) - identyfikator pytania, do którego należy opcja
 - content - treść opcji odpowiedzi
 - is_correct - czy opcja jest poprawną odpowiedzią
- **UserProgress:** śledzi postępy użytkowników w kursach (zabezpieczenie przed ponownym zdobywaniem expa)
 - id (PK) - unikalny identyfikator postępu
 - user_id (FK) - identyfikator użytkownika
 - course_id (FK) - identyfikator kursu
 - completion_date - data i czas ukończenia kursu
- **Post:** posty na forum
 - id (PK) - unikalny identyfikator postu
 - user_id (FK) - identyfikator autora postu
 - title - tytuł postu
 - content - treść postu
 - creation_date - data i czas utworzenia postu
- **Tag:** tagi przypisane do postów
 - id (PK) - unikalny identyfikator tagu
 - name - nazwa tagu
- **Comment:** komentarze do postów na forum
 - id (PK) - unikalny identyfikator komentarza
 - post_id (FK) - identyfikator postu, do którego należy komentarz
 - parent_comment_id (FK, nullable) - identyfikator komentarza nadrzędnego (jeśli to odpowiedź)
 - user_id (FK) - identyfikator autora komentarza
 - content - treść komentarza
 - creation_date - data i czas utworzenia komentarza
- **Reaction:** tabela reakcji (upvoty/reakcje emoji) do postów i komentarzy
 - id (PK) - unikalny identyfikator reakcji
 - reaction_type - typ reakcji (np. upvot, emoji)
- **UserStreak:** do przechowywania danych o długości streaków użytkowników¹
 - user_id (PK, FK) - identyfikator użytkownika
 - best_streak - aktualny streak użytkownika (najdłuższy)
 - begin_date - data rozpoczęcia aktualnego streaku

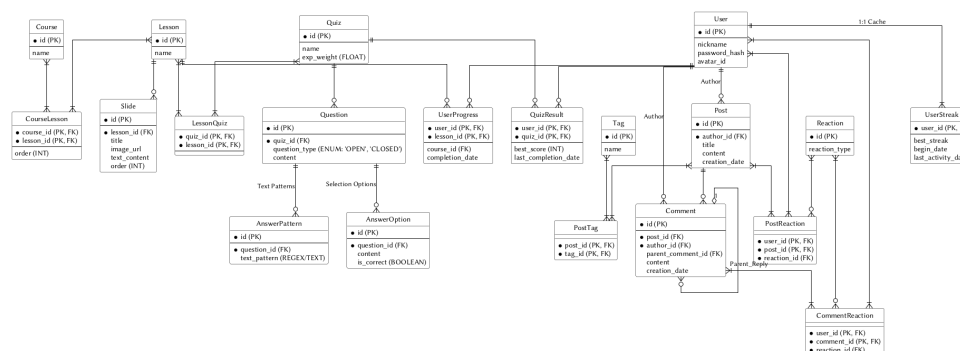
¹można by rozważyć czy nie przechowywać to w tabeli User, ale wydaje się to bardziej czytelne

- `last_activity_date` - data ostatniej aktywności użytkownika

Oraz wymagane tabele łączące:

- **QuizResult**: przechowuje wyniki quizów użytkowników
 - `user_id (PK, FK)` - identyfikator użytkownika
 - `quiz_id (PK, FK)` - identyfikator quizu
 - `best_score` - najlepszy wynik uzyskany w quizie
 - `last_completion_date` - data i czas ukończenia quizu
- **CourseLesson**: łącząca kursy z lekcjami
 - `course_id (PK, FK)` - identyfikator kursu
 - `lesson_id (PK, FK)` - identyfikator lekcji
 - `order` - kolejność lekcji w kursie
- **LessonQuiz**: łącząca lekcje z quizami
 - `lesson_id (PK, FK)` - identyfikator lekcji
 - `quiz_id (PK, FK)` - identyfikator quizu
- **PostTag**: łącząca posty z tagami
 - `post_id (PK, FK)` - identyfikator postu
 - `tag_id (PK, FK)` - identyfikator tagu
- **PostReaction**: łącząca posty z reakcjami
 - `post_id (PK, FK)` - identyfikator postu
 - `reaction_id (PK, FK)` - identyfikator reakcji
 - `user_id (PK, FK)` - identyfikator użytkownika, który dodał reakcję
- **CommentReaction**: łącząca komentarze z reakcjami
 - `comment_id (PK, FK)` - identyfikator komentarza
 - `reaction_id (PK, FK)` - identyfikator reakcji
 - `user_id (PK, FK)` - identyfikator użytkownika, który dodał reakcję

Poniżej znajduje się diagram ERD przedstawiający strukturę bazy danych oraz relacje między tabelami.



Rysunek 1: Diagram ERD bazy danych

1.1.2 Funkcje, Triggery

Zaplanowaliśmy również kilka funkcji oraz triggerów w bazie danych, które będą automatyzowały pewne procesy:

- Funkcja do aktualizacji streaku użytkownika w tabeli **UserStreak**. Przy każdym logowaniu należy sprawdzić datę ostatniej aktywności i jeżeli było to dzień wcześniej, zwiększyć aktualny streak o 1. Jeżeli nie było, to ustawić datę `begin_date` oraz `last_activity_date`

na dzisiejszą datę i zresetować.

Przy wykonaniu quizu należy ustawić `last_activity_date` na dzisiejszą datę i sprawdzić czy streak jest dłuższy niż `best_streak`, jeżeli tak to zaktualizować.

- Trigger po dodaniu rekordu do tabeli **UserProgress**, który będzie nadawał użytkownikowi odpowiednią ilość expa na podstawie wagi kursu.
- Odpowiednio utrzymywać dane w tabelach związanych z **Comment/Post** przy usuwaniu postów/komentarzy (kaskadowe usuwanie lub ustawianie na NULL w przypadku komentarzy nadrzędnych)

1.2 Permisje

Aby zabezpieczyć aplikację przed nieautoryzowanym dostępem, wprowadzimy system permisji. Użytkownicy będą podzieleni na dwie grupy:

- **Użytkownicy zarejestrowani**² - będą mieli dostęp do kursów, quizów oraz forum, nagród innych użytkowników
- **Administratorzy** - będą mieli dostęp do interfejsu administratora, gdzie będą mogli zarządzać kursami, quizami oraz użytkownikami

Dodatkowo, jeżeli użytkownik chciałby edytować/usunąć swój post/komentarz na forum, będzie wymagane sprawdzenie czy jest on autorem danego postu/komentarza.

1.3 Endpointy API

2 Interfejs administratora

Django posiada wbudowany interfejs administratora, który pozwala na zarządzanie bazą danych bez konieczności tworzenia własnego panelu. W naszym projekcie wykorzystamy ten interfejs do zarządzania w szczególności kursami oraz quizami.

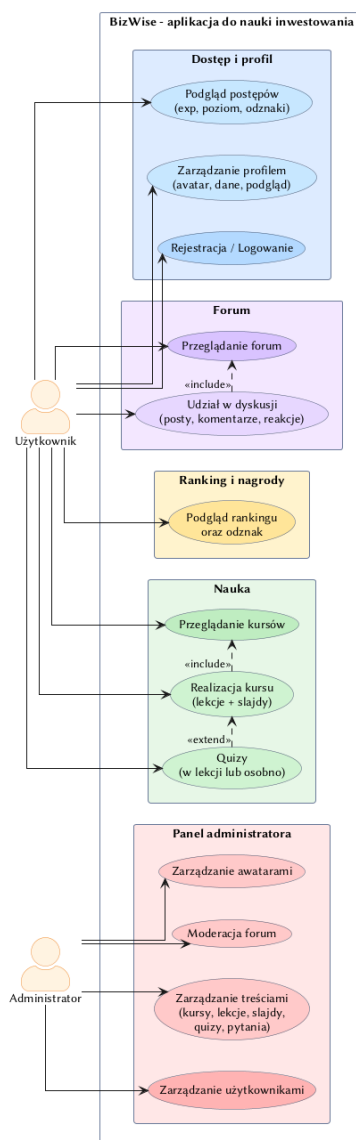
Administrator będzie miał możliwość tworzenia oraz edytowania poprzez interfejs administratora:

- tworzenie lekcji na podstawie dodawania slajdów z tekstem, ewentualnie obrazkami
- tworzenie quizów na podstawie pytań otwartych (wartości numeryczne) oraz zamkniętych.
- dodawanie avatarów użytkowników (userzy mogą wybierać avatary z puli dostępnych w serwisie)

3 Diagramy

Poniżej przedstawiamy diagramy przedstawiające strukturę projektu:

²restrakcja poprzez nickname i hasło



Rysunek 2: Diagram przypadków użycia